

Compose Navigation Architecture Guide

Model: google/gemma-4-e4b Created: 4/27/2026, 6:11:36 PM

User

Tell us about navigation in kotlin compose

Assistant

Of course! The topic of navigation in Jetpack Compose is one of the most important and frequently used topics. challenging topics, because it requires a change of mindset: we move from an imperative approach (Activity A calls Activity B) to a declarative approach (UI depends on the current state).

If we talk about "Pro" navigation, then it's not just about calling `NavController.navigate("route")`, and about the architectural approach to managing app status via the navigation graph and compliance with best practices MVVM/MVI.

Here's a detailed overview of key concepts, tools, and advanced patterns.

Fundamental Tools (The Basics)

It all starts with the `androidx.navigation: navigation-compose` library.

1. NavController

This is the main navigation controller. It is responsible for tracking the current state, managing the screen stack and performing the transitions themselves.

```
// Getting an instance of the controller in the Composable scope
val navController = rememberNavController()
```

2. NavHost

This is the host component, which knows about all possible routes (Routes) and displays the corresponding Composable based on the current state `navController`.

```
@Composable
fun AppNavHost(navController: NavHostController) {
    NavHost(
        navCount = 1, // Here you need to specify
        the home screen startDestination = "home",
        navController = navController
    ) { backStackEntry ->
        composable("home") { HomeScreen(onNavigateToDetails = { ...

        >composable("details/{itemId}") { backStackEntry ->
            // Обработка аргументов: получение ID из пути
            val itemId =
                backStackEntry.arguments?.getString("itemId")?: ""
            DetailScreen(itemId = itemId)
```

<Advanced Concepts (The " Pro " Level)

At the expert level, you should not think about traffic, but about the state of the app.

1. Managing arguments (Typed Arguments)

Instead of passing the ID as a string and parsing it on each screen, use structured arguments. Compose Navigation allows you to work with data types in the path:

```
>// 1. Определение маршрута с типом параметра
composable(
    route = "product/{productId}/{category}",
    arguments = listOf(navArgument("productId") { type =
NavType.StringType })
) { backStackEntry ->
    // 2. Безопасное получение аргумента нужного типа
    val productId: String =
        backStackEntry.arguments?.getString("productId")?: ""
```

```
<ProductScreen(productId = productId)
```

```
>// Навигация:  
navController.navigate ("product/$id/electronics")
```

<2. Using Router (Central Dispatcher)

The most common mistake beginners make is when every screen knows how to go to next screen. Correct architecture: create a separate dispatcher class (Router), which will be responsible for all traffic transitions and know about all possible routes.

Example: Instead of passing the `onNavigateToDetails` callback to `HomeScreen`:

```
>// Плохо: HomeScreen сам вызывает навигацию  
@Composable  
fun HomeScreen(navController: NavController) {  
    Button(onClick = { navController.navigate("details/$id") }) {}
```

<Use Router (the Mediator/Coordinator pattern):

Create an `AppRouter` or pass a handler function instead of a `NavController` from a parent component that knows about the overall application context. It makes screens should be as independent of the navigation mechanism as possible.

3. Back Stack Management

Sometimes it is necessary that when you go back, the user gets not only to the previous screen, but also some data was saved or some actions were performed (for example, updating the list).

- `popUpTo`: Allows you to "unload" multiple screens from the stack.

```
// Go to the profile, completely removing Home Screen  
from the stack navController.navigate ("profile") {  
    popUpTo("home") { inclusive = true } // Deleting everything up to  
    'home'
```

- * `launchSingleTop`: Ensures that if you try to navigate to the screen, which is already on the stack, you will not create a new copy of this screen.

4. Deep Linking

This is critical for integration with external systems (Brauseyandex. direct, push- notifications). Compose Navigation allows you to link a URL to a specific one on the route:

- 1. In nav_graph.xml or code: You declare the link scheme.
- 2. Processing in NavHost: Navigation automatically processes incoming links and navigates to the desired screen, as if the user clicked a button inside applications.

5. State Restoration

When you leave the screen (Screen A) and then return to it from Screen B, data may be lost.

Decision:

- 1. Save state in the ViewModel: Move all mission-critical items the data in the ViewModel is bound to the lifecycle of this Composable. Even if Compose destroys and restores the Composable, the ViewModel remains alive.
- 2. Using StateFlow / LiveData: Use reactive streams for transfer data from the business logic (ViewModel) directly to the UI, without relying on navigation parameters.



Summary table: "Pro" vs. Beginner

Aspect	The beginner's Approach (Beginner)	Professional approach (Pro/Architectural)
Broadcast data source	Stream arguments in the path (details/{id}), primitives.	Using ViewModel as the source of truth. Arguments — only for IDs.
Management click-throughs	Each Composable calls navigation itself (spaghetti code).	Central Router/Coordinator, which encapsulates all the logic click-throughs and knows all routes.
Condition of the screen	Relies only on parameters backStackEntry.	Uses the ViewModel to save states regardless of the life cycle the Composable loop.
Processing errors	No processing or the simplest try/catch methods.	Implementing a centralized system error handling (e.g., Result wrapper) and path handlers.

Aspect	The beginner's Approach (Beginner)	Professional approach (Pro/Architectural)
Testing	Difficult to test navigation.	Navigation Graph and Router easily underthey are given to unit tests, since the logic of click-throughs are placed in the class.